

Noetia — Documento de Proceso de Desarrollo

Versión 1.0 | Mayo 2026

1. Visión General

Este documento describe el proceso de desarrollo end-to-end utilizado para construir Noetia en 12 meses: desde el desarrollo local hasta la revisión de código, pruebas automatizadas, despliegue CI/CD y respuesta a incidentes. El proceso está diseñado para un equipo pequeño (4 personas) que debe hacer releases frecuentes sin sacrificar calidad ni estabilidad en producción.

2. Estructura del Repositorio

Noetia utiliza un **monorepo** alojado en GitHub. Todos los servicios — API, web, móvil, worker, generación de imágenes — viven en un único repositorio bajo `services/`. Esto elimina el drift de versiones entre servicios y hace que los cambios entre servicios sean atómicos.

```
noetia/
├── .github/
│   └── workflows/
│       ├── ci.yml           # Lint + test + build en cada PR
│       └── cd.yml          # Deploy a producción en push a main
├── services/
│   ├── api/                # Backend NestJS
│   ├── web/                # App web Next.js
│   ├── mobile/             # React Native (Expo)
│   ├── image-gen/          # Microservicio Python Flask
│   └── worker/              # Procesador de jobs BullMQ
├── infra/                  # Scripts de configuración del servidor, config de Traefik
├── docs/                   # Documentación del proyecto
├── docker-compose.yml      # Desarrollo local
├── docker-compose.prod.yml # Overlay de límites de recursos para producción
├── docker-compose.server.yml # Despliegue en producción
└── CLAUDE.md               # Guía del desarrollador (documento principal de
onboarding)
```

3. Flujo de Trabajo Git

Estrategia de branching

Noetia usa **desarrollo trunk-based** con una única rama de larga vida `main` . Las ramas de funcionalidad son de vida corta y se fusionan directamente a `main` vía pull request.

```
main ──────────────────────────── (producción)
    ↗ feature/clubs-backend ↘
    ↗ fix/audio-streaming   ↘
    ↗ chore/migration-053   ↘
```

Por qué trunk-based: En un equipo de 4 personas, las ramas de larga vida crean conflictos de merge innecesarios y retrasan la integración. Cada merge a `main` dispara un auto-deploy, por lo que la codebase siempre está en un estado desplegable.

Formato de mensaje de commit

```
<tipo>: <descripción corta>

Tipos: feat | fix | chore | docs | ci | refactor
```

Ejemplos:

```
feat: add phrase-anchored club discussions
fix: clamp heartbeat phraseDelta to zero to prevent negative stats
chore: add migration 053 – reading_stats table
docs: update CLAUDE.md with Whisper sync map procedure
ci: add concurrency group to CD to prevent parallel deploy races
refactor: extract TokenService from AuthService
```

Disciplina de commits

- **Un cambio lógico por commit.** Nunca agrupar cambios no relacionados.
- **Commit tras cada tarea completada.** Nunca dejar trabajo completado sin commitear de un día para otro.
- **Push inmediatamente después de cada commit.** Previene pérdidas de trabajo y mantiene la rama remota actualizada.

Pull requests

Todos los cambios pasan por un pull request, incluso el trabajo individual. Los PRs sirven como:

- Una verificación forzada de auto-revisión antes del merge
- El disparador del CI (lint + test + build)
- Un registro de auditoría para desarrolladores futuros

Plantilla de descripción de PR:

```
## Qué
[Un párrafo describiendo qué cambió]

## Por qué
[Por qué se necesita este cambio – referencia de ticket/sprint]

## Plan de prueba
[Cómo verificar que funciona – qué hacer clic, qué verificar]
```

4. Desarrollo Local

Prerequisitos

Herramienta	Versión	Notas
-------------	---------	-------

Docker	24+	Todos los servicios corren en Docker
Docker Compose	v2	docker compose (sin guion)
Node.js	20 LTS	Solo necesario si se corre api/web fuera de Docker
Python	3.11+	Solo necesario si se corre image-gen fuera de Docker

Iniciando el entorno de desarrollo

```
# Clonar el repositorio
git clone https://github.com/carloskfe/noetia.git
cd noetia

# Copiar y completar las variables de entorno
cp .env.example .env
# Editar .env – ver sección Variables de Entorno en CLAUDE.md

# Iniciar todos los servicios
docker compose up -d

# Verificar que todos los contenedores están corriendo
docker compose ps
```

Servicios disponibles tras el inicio:

Servicio	URL
Web (Next.js)	http://localhost:3000
API (NestJS)	http://localhost:4000
Consola MinIO	http://localhost:9001
Mailhog (preview de email)	http://localhost:8025
Meilisearch	http://localhost:7700

Hot reload

Los directorios fuente están montados como volúmenes Docker. Los cambios en archivos TypeScript/TSX/Python toman efecto sin reconstruir contenedores. Los cambios en `package.json`, `Dockerfile` o `tsconfig.json` requieren:

```
docker compose up -d --build <servicio>
```

Ejecutar migraciones en desarrollo

```
docker compose exec api npm run migration:run
```

5. Estrategia de Pruebas

Filosofía

Cada servicio tiene pruebas unitarias. Cada archivo de servicio tiene un archivo de prueba correspondiente. No hay excepciones. El conjunto de pruebas es el mecanismo de confianza principal que permite despliegues diarios a producción sin una fase de QA manual.

Umbral de cobertura

Todos los servicios imponen **≥ 80% de cobertura**. Las puertas de cobertura corren como parte del CI — un PR que baje la cobertura por debajo del 80% fallará la verificación de CI y no podrá hacer merge.

Estructura de archivos de prueba

Las pruebas espajan la estructura fuente exactamente:

```
services/api/
├─ src/
│   └─ stats/
│       ├── stats.service.ts
│       └─ stats.controller.ts
└─ tests/
    └─ unit/
        └─ stats/
            ├── stats.service.spec.ts
            └─ stats.controller.spec.ts
```

Reglas de aislamiento de pruebas

- **Simular todas las dependencias externas:** Ninguna prueba toca una base de datos real, Redis, MinIO o una API externa.
- **Sin estado compartido:** Cada caso de prueba es completamente independiente. El `beforeEach` reinicia todos los mocks.
- **Usar `jest.resetAllMocks()` en `beforeEach` — no `clearAllMocks()`.** Solo `resetAllMocks` limpia la cola de `mockResolvedValueOnce`, previniendo el sangrado de valores de mock entre pruebas.
- **Fechas dinámicas en pruebas:** Nunca hardcodear strings de fecha. Usar un helper como `utcDate(daysAgo)` que computa relativo a la fecha real de ejecución de la prueba.

Comandos para ejecutar pruebas

```
# API — desde la raíz del repo
docker compose exec api npm run test
docker compose exec api npm run test:cov

# Web
docker compose exec web npm run test
docker compose exec web npm run test:cov

# Generación de imágenes (Python)
```

```
docker compose exec image-gen pytest
docker compose exec image-gen pytest --cov=. --cov-report=term-missing
```

6. Pipeline CI/CD

CI — Integración Continua

Disparador: Cada pull request a `main`, cada push a `main`.

Archivo: `.github/workflows/ci.yml`

Pasos:

1. Checkout del código
2. Configurar Node.js 20
3. Instalar dependencias (api, web, worker, mobile)
4. Lint de todos los servicios TypeScript (ESLint)
5. Type-check de todos los servicios TypeScript (tsc --noEmit)
6. Ejecutar pruebas unitarias con cobertura (api, web, worker)
7. Configurar Python 3.11
8. Instalar dependencias Python (image-gen)
9. Ejecutar pytest con cobertura (image-gen)
10. Compilar imágenes Docker (api, web, image-gen, worker)

Un PR no puede hacer merge si algún paso falla.

Tiempo de ejecución: ~6–8 minutos en el tier gratuito de GitHub Actions.

CD — Despliegue Continuo

Disparador: Push a `main` (tras pasar el CI).

Archivo: `.github/workflows/cd.yml`

Pasos:

1. SSH al servidor de producción (puerto 222, secreto `DEPLOY_SSH_KEY`)
2. `cd /opt/noetia && git pull origin main`
3. `docker compose --env-file .env.production \`
 `-f docker-compose.server.yml \`
 `up -d --build`
4. `docker compose exec -T api npm run migration:run:prod`
5. `docker image prune -f`

Control de concurrencia: El workflow de CD usa un grupo de concurrencia (`deploy-production`) con `cancel-in-progress: false`. Esto serializa los deploys — si llegan dos pushes simultáneamente, el segundo espera a que el primero se complete en lugar de correr en paralelo.

```
concurrency:
  group: deploy-production
  cancel-in-progress: false
```

Limpieza de contenedores: Tras el deploy, el workflow ejecuta una limpieza dinámica que elimina los contenedores detenidos con el patrón `noetia-*`.

Duración del deploy: ~4-6 minutos incluyendo las compilaciones de imágenes.

Secretos de despliegue

Todos los secretos se almacenan en los ajustes del repo de GitHub (Settings → Secrets → Actions):

Secreto	Propósito
DEPLOY_SSH_KEY	Clave SSH privada para desplegar al servidor de producción
SENTRY_AUTH_TOKEN	Usado durante el build para subir source maps a Sentry

Las variables de entorno de producción (`DATABASE_URL` , `STRIPE_SECRET_KEY` , etc.) se almacenan en `.env.production` en el servidor en `/opt/noetia/.env.production` — nunca en GitHub.

7. Proceso de Migración de Base de Datos

Las migraciones son la parte más sensible del proceso de despliegue. Una migración mal hecha puede corromper datos de producción.

Crear una migración

```
# Generar una nueva migración desde cambios en entidades
docker compose exec api npm run migration:generate -- src/migrations/<timestamp>-<Descripcion>

# 0 escribir una migración manual
# Crear src/migrations/<timestamp>-<Descripcion>.ts
# Implementar los métodos up() y down()
```

Convención de nombres de migración

`<timestamp-13-dígitos>-<DescripcionEnPascalCase>.ts`

Ejemplos:
1748000000000-CreateReadingStats.ts
1748100000000-AddPrivacySettings.ts
1748200000000-AddWeeklyGoals.ts

Checklist de revisión de migración

Antes de hacer merge a cualquier migración:

- ☐ La migración es aditiva (agrega columnas/tablas), no destructiva
- ☐ Si se elimina una columna: el código de la aplicación ya no la referencia y fue desplegado primero
- ☐ Las claves foráneas tienen comportamiento `ON DELETE` explícito especificado
- ☐ Cualquier nuevo índice tiene un nombre significativo (`idx_<tabla>_<columna>`)

- ☐ El método `down()` está implementado y probado localmente
- ☐ La migración se ejecutó localmente: `docker compose exec api npm run migration:run`

Ejecución de migraciones en producción

Las migraciones se ejecutan automáticamente durante el CD (paso 4 del pipeline). Si una migración falla, el pipeline de deploy falla y los contenedores anteriores continúan corriendo.

8. Definición de Done

Una funcionalidad o tarea está **Terminada** cuando todo lo siguiente es verdadero:

Código

- ☐ La funcionalidad funciona según los criterios de aceptación
- ☐ Sin errores TypeScript (`tsc --noEmit` pasa)
- ☐ Sin errores ESLint (`eslint .` pasa)

Pruebas

- ☐ El archivo de prueba unitaria existe en la ruta espejada bajo `tests/unit/`
- ☐ Todas las pruebas pasan (`npm run test / pytest`)
- ☐ La cobertura para el servicio modificado es $\geq 80\%$
- ☐ Las pruebas cubren: camino feliz, casos borde y escenarios de error
- ☐ Ninguna prueba depende de una base de datos real o llamada de red externa

Revisión

- ☐ Pull request creado con una descripción que explica qué y por qué
- ☐ Al menos un miembro del equipo ha revisado y aprobado
- ☐ Todas las verificaciones de CI pasan (lint, type-check, pruebas, build)

Despliegue

- ☐ Fusionado a `main`
 - ☐ Pipeline CD ejecutado exitosamente
 - ☐ El PM ha verificado la funcionalidad en producción contra los criterios de aceptación
-

9. Estándares de Revisión de Código

Qué verifican los revisores

Corrección:

- ¿La implementación coincide con los criterios de aceptación?
- ¿Se manejan los casos borde (null, lista vacía, acceso concurrente)?
- ¿Los errores se devuelven con el código de estado HTTP correcto?

Seguridad:

- Sin valores proporcionados por el usuario concatenados en SQL (usar consultas parametrizadas de TypeORM)
- Sin datos sensibles logueados (tokens, contraseñas)

- Los nuevos endpoints están protegidos por `JwtAuthGuard` a menos que sean explícitamente públicos

Consistencia:

- Sigue los patrones existentes (estructura de módulos, validación de DTOs, acceso al repositorio)
- Las nuevas columnas/tablas siguen la convención de nombres (camelCase en entidad, snake_case en BD vía TypeORM)
- Las pruebas siguen la convención de estructura espejada

Alcance:

- Sin funcionalidades añadidas más allá de lo que especifica el ticket
- Sin abstracciones especulativas o código de "quizás lo necesitemos después"

Tiempo de respuesta

Las revisiones deben completarse dentro de 1 día hábil. Los PRs no permanecen sin revisión por más de 24 horas.

10. Gestión de Entornos

Tres entornos

Entorno	Cómo ejecutar	Base de datos	Auth
Desarrollo	<code>docker compose up</code>	PostgreSQL local	JWTs de dev, Mailhog
Preview	<code>EAS build --profile preview</code> (móvil)	Producción (datos de prueba, solo lectura)	Auth real
Producción	Auto-desplegado vía CD	PostgreSQL de producción	Auth real, Stripe real

No hay servidor de staging — el único VPS Contabo corre producción. Los desarrolladores validan las funcionalidades localmente y confían en el conjunto de pruebas de CI para tener confianza antes del merge.

11. Respuesta a Incidentes

Niveles de severidad

Nivel	Descripción	Ejemplo	Tiempo de respuesta
P1	Producción caída	API inalcanzable, autenticación rota	Inmediato
P2	Funcionalidad rota	Lector no carga, pagos fallando	< 2 horas
P3	Degradado	Búsqueda lenta, errores de generación de imágenes	< 24 horas

Procedimiento de respuesta P1

1. **Confirmar** — verificar alerta Grafana, `docker ps`, `docker compose logs -f api`
2. **Identificar** — ¿es un deploy nuevo? Verificar `git log` para merges recientes
3. **Rollback si es causado por cambio de código:**

```
cd /opt/noetia
git revert HEAD --no-edit
git push
# Esperar a que CD redespliegue automáticamente (~5 min)
```

4. **Rollback si es causado por migración:** Restaurar desde backup diario (snapshot PostgreSQL a las 02:00 UTC)
5. **Comunicar** — publicar en Slack `#incidentes` con actualizaciones de estado cada 15 minutos
6. **Resolver** — una vez que el servicio es estable, escribir un reporte de incidente en Notion

Modos de fallo comunes y diagnóstico

API 502 / Traefik no puede alcanzar el contenedor:

```
docker ps # ¿Está el contenedor api saludable?
docker inspect noetia-api-1 --format "{{.State.Health.Status}}"
docker compose -f docker-compose.server.yml logs --tail 50 api
```

Migración falló en el deploy:

```
docker compose -f docker-compose.server.yml logs --tail 20 api
# Buscar el error de migración – corregir la migración, hacer push, dejar que CD
redespliegue
```

Alerta de uso de disco (>80%):

```
df -h # Verificar disco
docker image prune -a # Limpiar imágenes sin usar
docker system prune # Más agresivo (también detiene contenedores sin uso)
```

Acceso al servidor

```
# SSH (puerto 222 – cambiado del 22 predeterminado)
ssh -p 222 root@84.247.140.175

# Ver logs en vivo
docker compose -f docker-compose.server.yml logs -f api
docker compose -f docker-compose.server.yml logs -f web

# Acceder a Grafana (túnel SSH desde máquina local)
ssh -p 222 -L 3001:localhost:3001 root@84.247.140.175
# Luego abrir http://localhost:3001

# NUNCA pegar contenido multilínea vía SSH – usar nano o codificación base64
# Ver CLAUDE.md §Operaciones críticas del servidor para más detalles
```

12. Prácticas de Documentación

CLAUDE.md — Guía viva del desarrollador

CLAUDE.md es el documento principal de onboarding y operacional del equipo. Vive en la raíz del repositorio y se actualiza siempre que:

- Se agrega un nuevo servicio o módulo
- Se aplica una nueva migración
- Cambia un procedimiento de infraestructura
- Se descubre una lección difícilmente aprendida (p.ej., corrupción de pegado multilínea por SSH)

Es el primer documento que lee un nuevo desarrollador. Siempre debe estar lo suficientemente actualizado para que un desarrollador pueda configurar un entorno de trabajo desde cero en menos de 30 minutos.

Política de comentarios en código

Los comentarios se escriben solo cuando el POR QUÉ no es obvio:

- Una restricción o invariante oculta
- Una solución provisional para un bug específico de una librería externa
- Una optimización de rendimiento que parece una pesimización

Los comentarios que explican QUÉ hace el código no se escriben — un buen naming lo hace obvio.

Descripciones de PR

Las descripciones de PR sirven como el registro principal de por qué se realizó un cambio. Deben explicar:

- El problema que se está resolviendo
- Por qué se eligió este enfoque específico (si hay alternativas)
- Cualquier riesgo o advertencia

Documento mantenido por el Ingeniero DevOps y el Desarrollador Backend. Última actualización: 25 de mayo de 2026.